

AI for Statistical Learning — Week 2

Week 2 - Assignment 1: Optimizing Linear Regression using Distribution Theory

Group 2: Carlos, Landon, and Elias.

2026-06-06

Introduction

Previously, we wrote our linear regression model as $y_i = \beta_0 + \beta_1 x_i + \epsilon_i$, where $\epsilon_i \sim N(0, \sigma^2)$.

Turn out, using the properties of the normal distribution, an equivalent specification is: $y_i \sim N(\beta_0 + \beta_1 x_i, \sigma^2)$.

So instead of using $\sum \epsilon_i^2$ as the loss function, we can also use the negative log likelihood of the normal distribution as the loss function.

Consider the following data generation process:

```
set.seed(0)
n <- 1000
x <- rnorm(n = n, mean = 0, sd = 2)
b0 <- -2; b1 <- 7.7
y <- rnorm(n, b0 + b1 * x, sd = 6)
```

The likelihood of the normal distribution for $y_i, \dots, y_n$ (where $n = 1000$) is:

$$L(\beta|y) = \frac{1}{(6\sqrt{2\pi})^n} \exp\left\{-\frac{1}{2 \cdot 6^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right\}$$

where $\beta = (\beta_0, \beta_1)$

Estimate β using the following steps:

1. Write down an expression of the negative log-likelihood (nll) - $-l(\beta|y) = -\log L(\beta|y)$
2. Use the **optim** function to minimize the nll and estimate β .
3. Confirm your answer by using the **dnorm** function instead of manually calculating the nll.
4. [BONUS] In the previous parts, we fixed $\sigma = 6$. Can you also try estimating it from the data as well? You will need to estimate $\theta = (\beta_0, \beta_1, \sigma)$ where the first components are unbounded but the last one is strictly positive.

i Answer 1

- The likelihood of the normal Distribution:

$$L(\beta|y) = \frac{1}{(\sigma\sqrt{2\pi})^n} \exp\left\{-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right\}$$

- The expression of the negative log-likelihood (nll):

$$nll = -l(\beta|y) = -\log L(\beta|y) \Rightarrow -l(\beta|y) = -\log\left(\frac{1}{(\sigma\sqrt{2\pi})^n} \exp\left\{-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right\}\right)$$

- Using some properties of logarithms, we obtain the following simplified expression for the nll:

$$nll = -l(\beta | y) = +n \log(\sigma\sqrt{2\pi}) + \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

- In our case, since $\sigma = 6$, and $n = 1000$, the expression becomes:

$$nll = 1000 \log(6\sqrt{2\pi}) + \frac{1}{72} \sum_{i=1}^{1000} (y_i - \beta_0 - \beta_1 x_i)^2.$$

i Answer 2

- Write the nll function to be minimized by `optim` and then minimize it:

```
nll_loss <- function(theta){  
  b0 = theta[1]  
  b1 = theta[2]  
  nll = n * log(6*sqrt(2*pi)) + sum((y - b0 - b1*x)^2)/72  
  return(nll)  
}
```

- Minimizing the nll function:

```
optim(c(0, 0), nll_loss)
```

```
$par  
[1] -2.151580  7.660794
```

```
$value  
[1] 3244.403
```

```
$counts  
function gradient  
      85      NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

i Answer 3

- Confirming the answer using the `dnorm` function instead of manually calculating the nll:

```
ll <- function(theta){  
  b0 = theta[1]  
  b1 = theta[2]  
  # Sum of the logs of the density at each point in the y vector  
  llik = sum(dnorm(y, mean = b0 + b1 * x, log = T))  
  # Negative log-likelihood to be minimized  
  return(-llik)  
}
```

- Minimizing the nll function:

```
optim(c(0, 0), ll)
```

```
$par  
[1] -2.151580  7.660794
```

```
$value  
[1] 20132.32
```

```
$counts  
function gradient  
      85      NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

Bonus Question

```
nll_Sig <- function(theta){  
  b_0 = theta[1]  
  b_1 = theta[2]  
  sigma = theta[3]  
  nll = n * log(sigma*sqrt(2*pi)) + sum((y - b_0 - b_1*x)^2)/(2*(sigma)^2)  
  return(nll)  
}
```

- Minimizing the nll function:

```
optim(c(0, 0, 1),  
      nll_Sig,  
      method = "L-BFGS-B",  
      lower = c(-Inf, -Inf, 1e-10),  
      upper = c(Inf, Inf, Inf))
```

\$par

[1] -2.149962 7.660663 6.199288

\$value

[1] 3243.316

\$counts

function gradient
18 18

\$convergence

[1] 0

\$message

[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EPSMCH"